RELATÓRIO - CRIAÇÃO E MANIPULAÇÃO DE POLÍGONOS EM OPENGL

Modelo organizado para preenchimento

Nome do aluno:	Matheus Henrique de Oliveira Garcia	**Matrícula:**	22450747
Disciplina:	Introdução a computação gráfica	**Professor(a):**	Zeca Pio
Turma:	CB01	**Data:**	17/04

1. Dados gerais do projeto

Título do programa:

OpenGL 2D Polygon - manipulação interativa de polígono regular em 2D.

Linguagem utilizada, bibliotecas e ferramentas:

Python 3.11, PyOpenGL, OpenGL/GLUT, Docker, noVNC, pytest, Black e Ruff.

Resumo do funcionamento do programa:

O programa abre uma janela OpenGL 2D e desenha um polígono regular a partir de um estado centralizado em uma dataclass imutável. Cada entrada do teclado altera esse estado, e a tela é redesenhada imediatamente com os novos atributos. O renderizador recalcula os vértices com seno e cosseno, limpa o frame anterior e desenha a figura com GL_POLYGON ou GL_LINE_LOOP, conforme o modo selecionado.

Objetivo da aplicação e o que ela permite modificar interativamente:

A aplicação tem como objetivo demonstrar a modelagem computacional de um polígono regular e a separação entre geometria e aparência visual. Ela permite alterar interativamente o número de vértices, a posição no plano, a cor de preenchimento, a cor da borda, a espessura da linha e o modo de exibição entre preenchido e contorno.

Estrutura geral do código e organização das principais funções:

O código foi dividido por responsabilidade. app/main.py inicializa a janela, registra os callbacks da GLUT e conecta entrada e renderização. app/core/state.py concentra os atributos do polígono e os limites da aplicação. app/core/actions.py implementa as mudanças de estado. app/core/polygon.py gera os vértices do polígono regular. app/input/keyboard.py faz o mapeamento das teclas. app/render/renderer.py configura a projeção ortográfica e envia os vértices ao OpenGL. A pasta tests valida estado inicial, ações do teclado e geração correta dos vértices.

2. Requisitos implementados

Descreva de forma breve como cada requisito foi implementado no seu programa.

Requisito	Como foi implementado	Observações
Criação do polígono a partir dos vértices	Os vértices são gerados pela função generate_regular_vertices(), que calcula um polígono regular a partir do centro, do raio e da quantidade de lados usando seno e cosseno.	Os vértices são criados em ordem anti-horária e o código exige no mínimo 3 vértices.

Exibição do polígono na tela	O método render() limpa o buffer, recalcula os vértices a partir do estado atual e desenha a figura com glBegin()/glEnd() e glVertex2f().	A janela usa projeção ortográfica fixa no intervalo de -1 a 1 nos eixos x e y.
Alteração do número de vértices	As teclas N e M chamam ações que incrementam ou decrementam vertex_count no estado da aplicação.	O valor é limitado entre 3 e 12 vértices.
Movimento do polígono	As setas alteram as coordenadas x e y do centro do polígono por meio de ações específicas de movimento.	O deslocamento é feito em passos de 0,05 e não altera a forma nem o raio.
Mudança da cor de preenchimento	A tecla C avança o índice de cor de preenchimento em uma paleta circular fixa com 8 cores.	A cor de preenchimento é aplicada com glColor3f() quando o modo está em preenchimento.

Mudança da cor da borda	A tecla V avança o índice de cor da borda em uma paleta circular independente da cor de preenchimento.	A cor da borda é usada quando a figura é desenhada em modo contorno.
Alteração da espessura da borda	A tecla B altera line_width de forma cíclica e o valor é aplicado com glLineWidth() antes do desenho do contorno.	A espessura varia de 1,0 até 10,0 e depois reinicia em 1,0.
Alternância entre preenchido e contorno	A tecla P alterna o enum DisplayMode entre FILLED e OUTLINE.	No modo FILLED o programa usa GL_POLYGON. No modo OUTLINE usa GL_LINE_LOOP.

3. Controles e interação

Informe as teclas, comandos ou interações usadas para controlar o polígono e seus atributos.

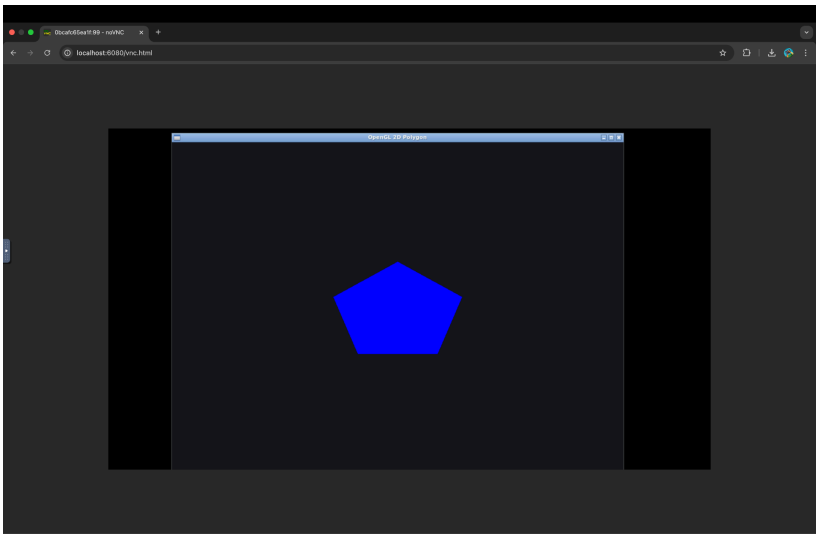
Ação	Tecla(s) usada(s)	Observações
Aumentar número de vértices	N	Aumenta um vértice por vez, até o máximo de 12.
Diminuir número de vértices	M	Reduz um vértice por vez, até o mínimo de 3.
Mover o polígono	↑ ↓ ← →	Move o centro em passos de 0,05 no plano 2D.

Mudar cor de preenchimento	C	Percorre uma paleta circular com 8 cores.
Mudar cor da borda	V	Percorre a mesma paleta, mas com índice separado.
Alterar espessura da borda	B	Aumenta a espessura do contorno e reinicia após 10,0.
Alternar preenchido/contorno	P	Troca entre GL_POLYGON e GL_LINE_LOOP.
Outra interação implementada	N, M, C, V, B e P em maiúsculo ou minúsculo	As teclas alfabéticas são normalizadas para minúsculas antes do mapeamento.

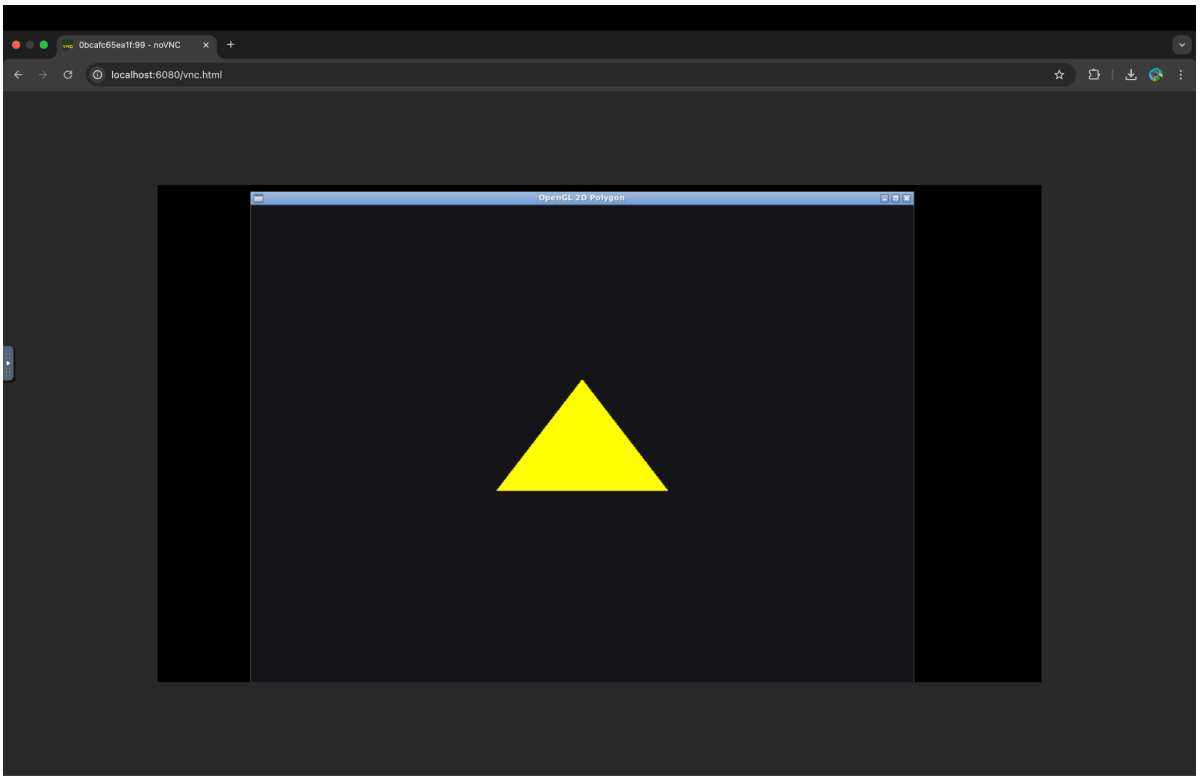
4. Capturas de tela

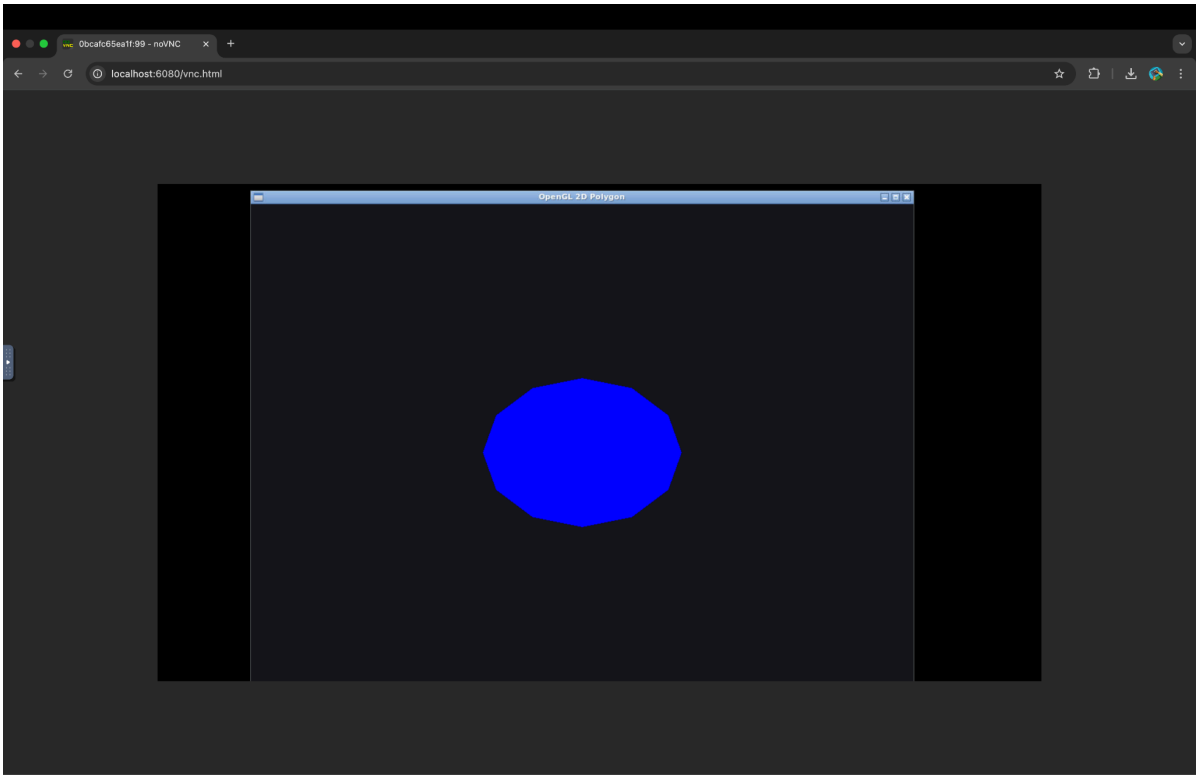
Insira imagens do programa em funcionamento com diferentes configurações.

Captura 1 - Estado inicial do polígono

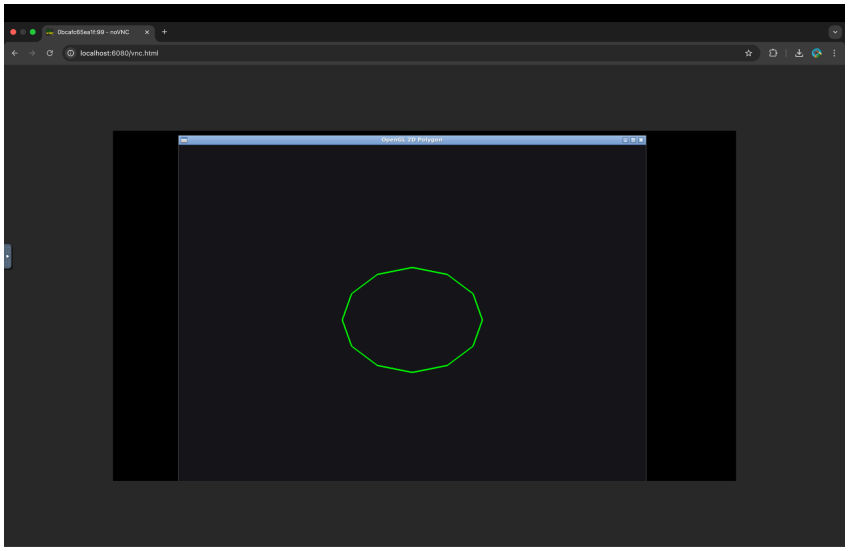


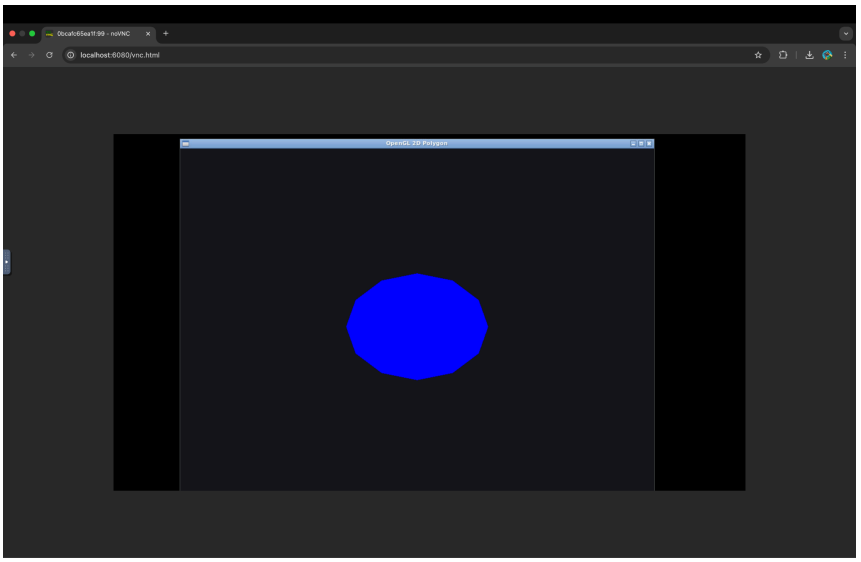
Captura 2 - Alteração do número de vértices

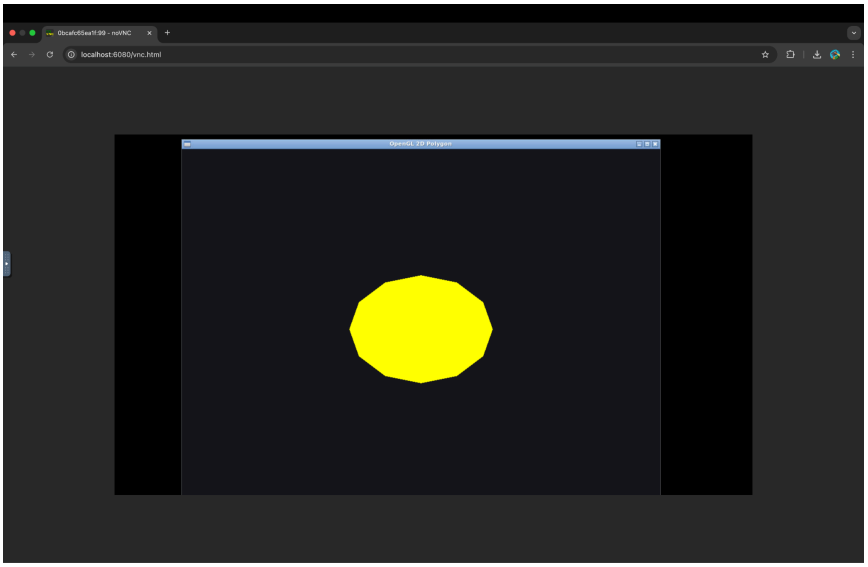




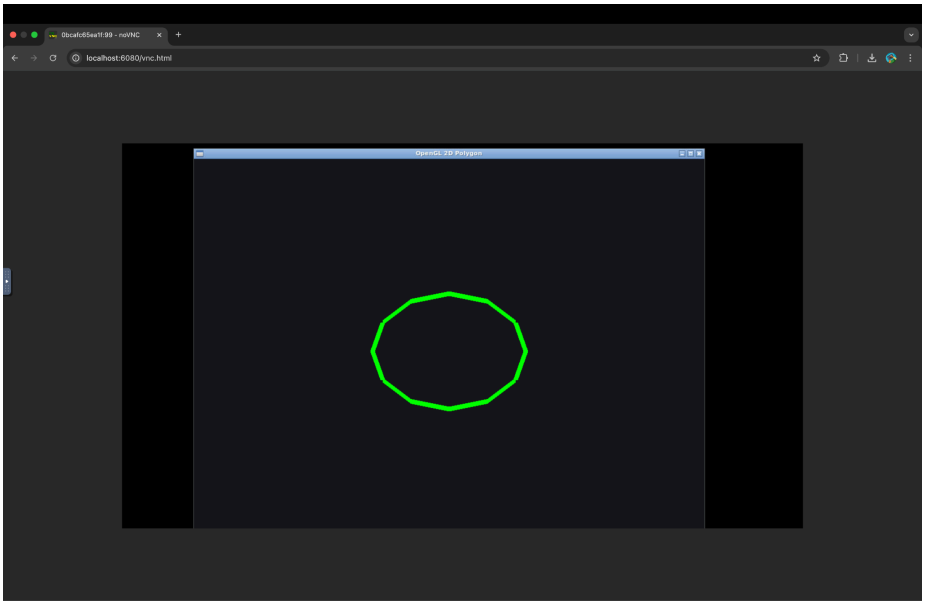
Captura 3 - Mudança de cor de preenchimento e/ou borda

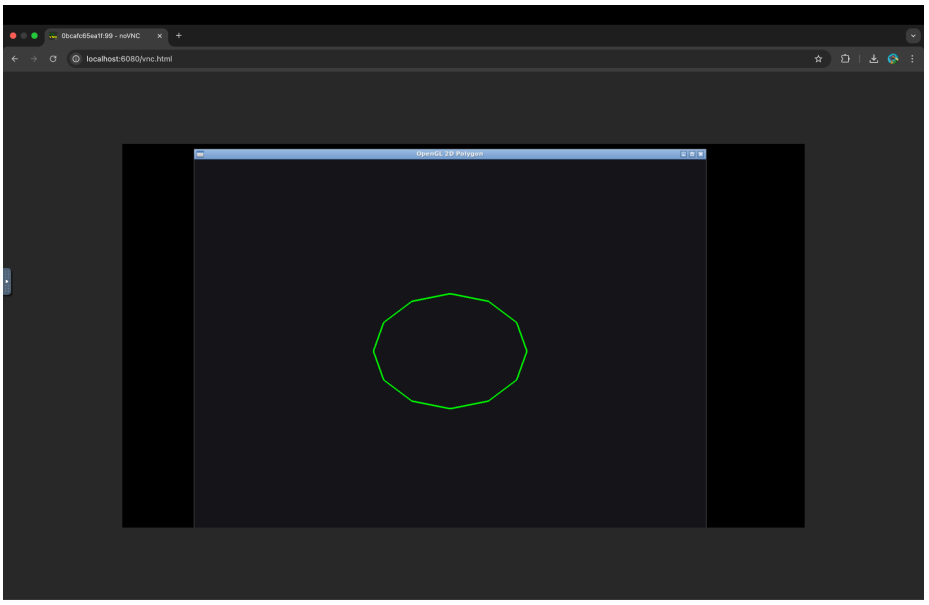




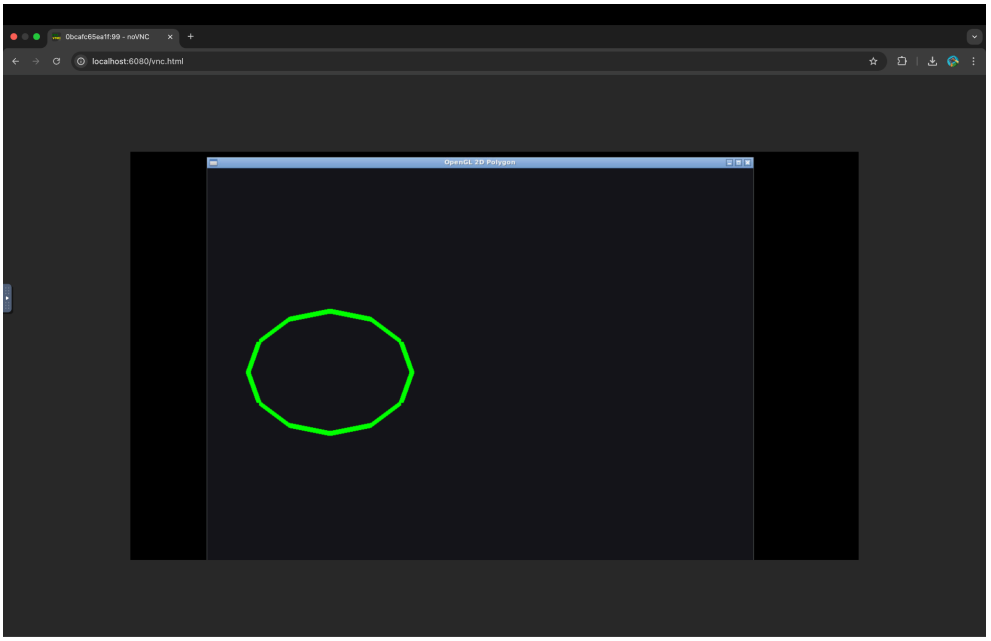


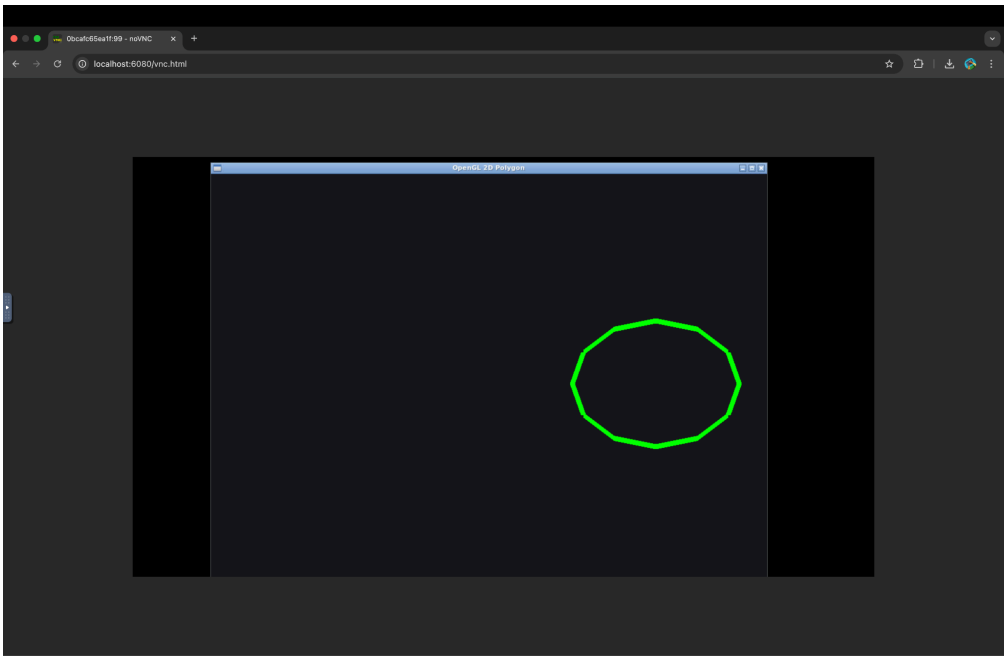
Captura 4 - Alteração da espessura da borda



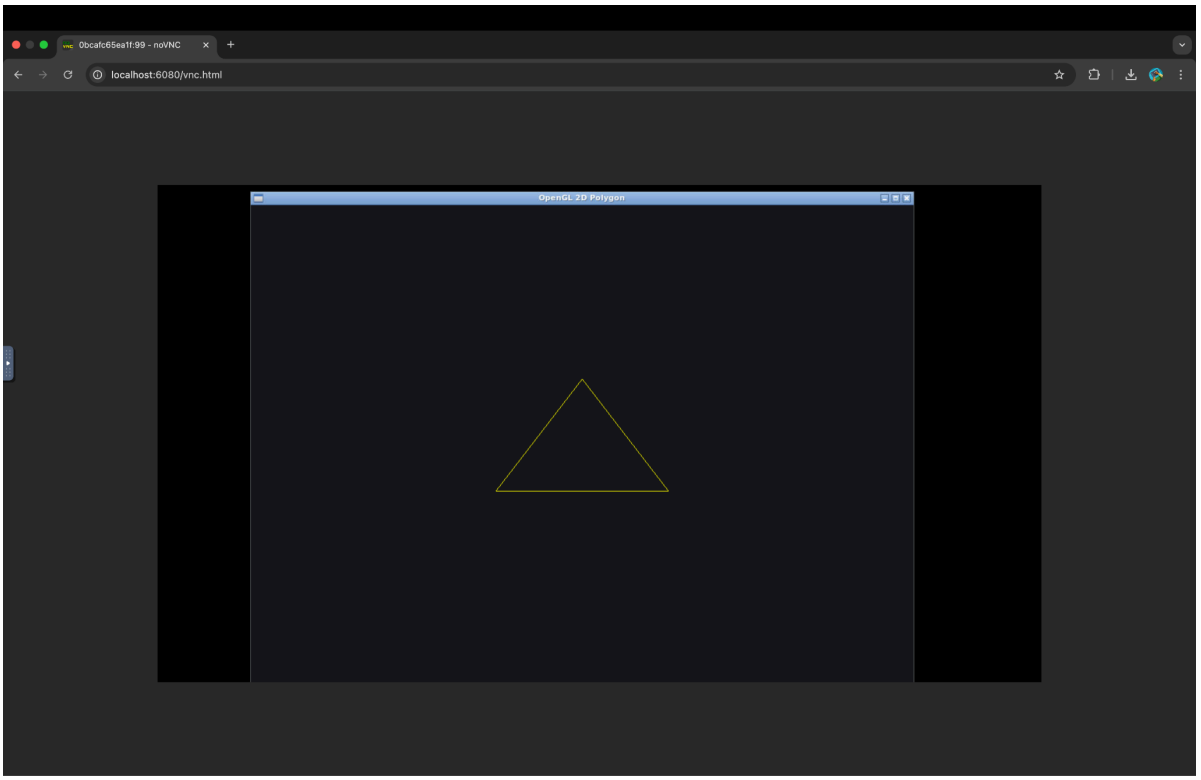


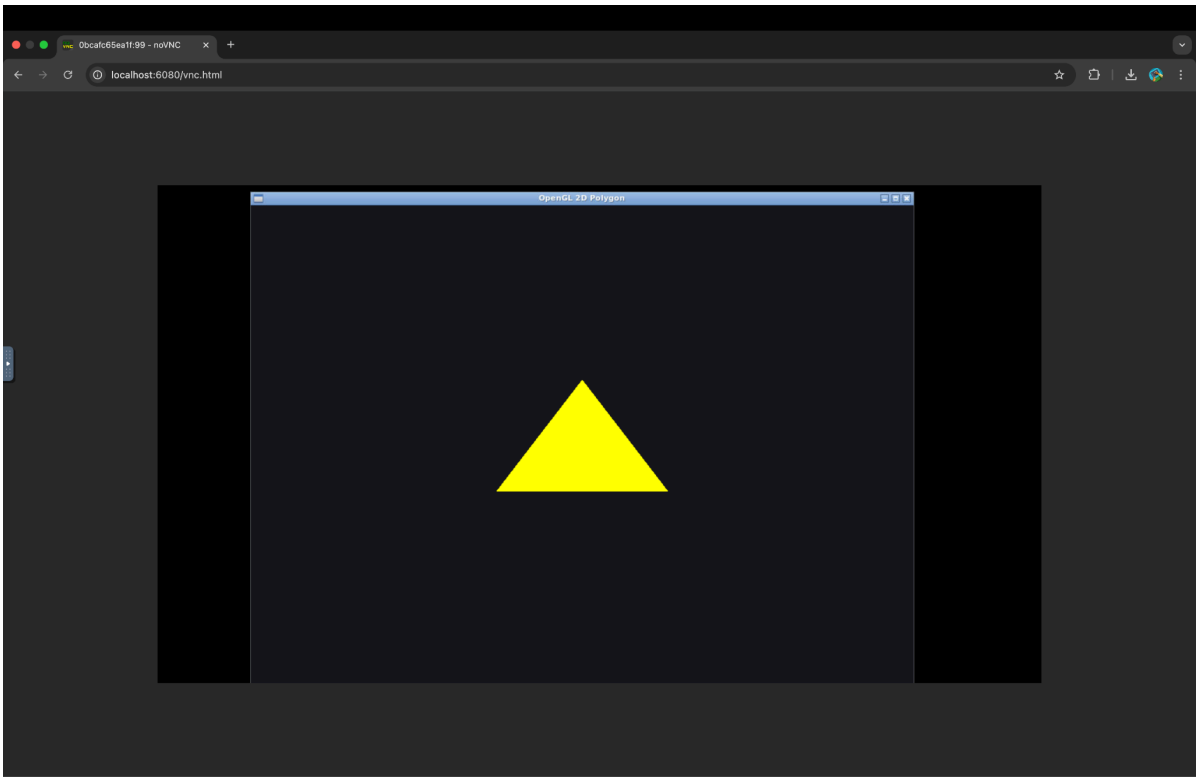
Captura 5 - Movimento do polígono na tela





Captura 6 - Alternância entre preenchido e contorno





5. Respostas às questões

Responda às perguntas abaixo com base no comportamento do seu programa e nos conceitos de OpenGL trabalhados na atividade.

1. O que acontece ao aumentar o número de vértices do polígono?

Resposta:

Ao aumentar o número de vértices, o polígono passa a ter mais lados e sua forma fica mais próxima de uma circunferência. No programa, esse valor cresce de forma controlada até o limite de 12 vértices.

2. Qual a diferença entre alterar a posição do polígono e alterar seus vértices?

Resposta:

Alterar a posição desloca o polígono inteiro no plano sem mudar sua forma, seu tamanho nem a relação entre os vértices. Alterar os vértices modifica a geometria da figura, porque muda os pontos que definem sua forma. Nesta implementação, o movimento atua apenas sobre o centro do polígono, por meio das coordenadas x e y.

3. Como a cor e a espessura da borda influenciam a visualização da figura?

Resposta:

A cor define o contraste e facilita a identificação visual da figura. A espessura da borda altera o destaque do contorno: linhas mais finas produzem um resultado mais discreto, enquanto linhas mais grossas deixam a forma mais marcada. Nesta implementação, a borda e sua espessura ficam visíveis no modo contorno.

4. Qual a função da primitiva GL_LINE_LOOP no OpenGL?

Resposta:

GL_LINE_LOOP desenha apenas o contorno do polígono. Ela liga cada vértice ao próximo e fecha a figura conectando automaticamente o último vértice ao primeiro.

5. Qual a diferença entre as primitivas GL_POLYGON e GL_LINE_LOOP?

Resposta:

GL_POLYGON desenha a área interna preenchida da figura. GL_LINE_LOOP desenha somente o contorno fechado. Em outras palavras, uma primitiva representa a superfície e a outra representa apenas a borda.

6. Por que um polígono precisa ter pelo menos três vértices?

Resposta:

Porque com menos de três vértices não existe uma área fechada. Um vértice representa apenas um ponto e dois vértices representam apenas um segmento de reta. O terceiro vértice é o mínimo necessário para formar uma superfície poligonal.

7. O que ocorre visualmente quando os vértices não são definidos em ordem adequada?

Resposta:

Quando os vértices não são definidos em uma ordem adequada, o contorno pode se cruzar e a figura pode parecer distorcida ou autointersectante. Em operações de preenchimento, isso também pode gerar resultados visuais inesperados. No código analisado, esse problema é evitado porque os vértices são gerados em ordem anti-horária.

8. Como o sistema de coordenadas influencia a posição do polígono na tela?

Resposta:

O sistema de coordenadas determina onde cada vértice será desenhado na janela. Nesta aplicação, a projeção ortográfica foi configurada para o intervalo de -1 a 1 em x e y. Assim, aumentar x move a figura para a direita, diminuir x move para a esquerda, aumentar y move para cima e diminuir y move para baixo. Valores fora desse intervalo podem tirar a figura parcial ou totalmente da área visível.

9. Qual a diferença entre atributos geométricos e atributos visuais em uma figura gráfica?

Resposta:

Atributos geométricos definem a estrutura da figura, como quantidade de vértices, coordenadas, centro e raio. Atributos visuais definem sua aparência, como cor de preenchimento, cor da borda, espessura da linha e modo de exibição. O código separa bem esses dois grupos dentro do estado do polígono e do renderizador.

10. O que acontece quando a espessura da linha (glLineWidth) é aumentada significativamente?

Resposta:

Quando glLineWidth é aumentada significativamente, o contorno fica mais espesso e mais evidente. Isso melhora a visibilidade da borda, mas também pode deixar a figura visualmente pesada e reduzir a percepção de detalhes mais finos. No programa, a espessura varia ciclicamente entre 1,0 e 10,0.

11. Como a alteração do número de vértices pode aproximar o polígono de uma circunferência?

Resposta:

Quanto maior o número de vértices de um polígono regular, menor fica o ângulo entre lados consecutivos e mais suave se torna seu contorno. Por isso, ao aumentar a quantidade de vértices, a figura se aproxima visualmente de uma circunferência.

12. Qual é o papel da função glColor na renderização do polígono?

Resposta:

A função glColor define a cor atual usada na renderização dos próximos vértices e primitivas desenhados. Nesta implementação, glColor3f() recebe a cor de preenchimento quando a figura é desenhada com GL_POLYGON e a cor da borda quando ela é desenhada com GL_LINE_LOOP.

13. O que acontece quando a cor de preenchimento e a cor do contorno são iguais?

Resposta:

Nesta implementação, o preenchimento e o contorno não são desenhados ao mesmo tempo, porque o programa alterna entre os modos preenchido e contorno. Por isso, usar a mesma cor para preenchimento e borda não produz diferença visual direta dentro da lógica atual. Em uma versão que desenhasse os dois simultaneamente, o contorno teria pouco contraste em relação ao interior da figura.

14. Como o uso do teclado contribui para a interatividade da aplicação gráfica?

Resposta:

O teclado torna a aplicação interativa porque permite alterar o estado do polígono em tempo real sem reiniciar o programa. Cada tecla está associada a uma ação específica, e após a mudança o código chama glutPostRedisplay() para redesenhar a tela imediatamente. Isso permite testar rapidamente diferentes configurações da figura.

15. Quais dificuldades você encontrou ao implementar a manipulação dinâmica dos atributos do polígono?

Resposta:

Uma dificuldade foi organizar a atualização dos atributos pelo teclado e redesenhar o polígono corretamente a cada alteração.

6. Observações finais

Comentários finais, melhorias futuras ou observações adicionais:

A aplicação atende ao núcleo da atividade com uma arquitetura modular, separação entre estado, entrada e renderização, e geração procedural dos vértices do polígono. O projeto também inclui testes automatizados para regras centrais do estado e do cálculo geométrico. Como melhorias futuras, seria possível desenhar preenchimento e contorno ao mesmo tempo, permitir edição manual dos vértices, exibir coordenadas na tela e suportar mais de um polígono.